Informe Tarea 2

Torneo de Komikan Concurrente

Asignatura: Sistemas Operativos
Integrantes: Valentina Díaz - Valentina Araya

1. Descripción del proyecto

El proyecto consiste en desarrollar un programa que permita simular partidas del juego Komikan de manera concurrente. Para esto se utilizan hilos que permiten ejecutar varias partidas al mismo tiempo, administrar jugadores y mesas, controlar los movimientos de las fichas y detectar cuando una partida termina.

Además, el sistema considera situaciones como el abandono de jugadores y la interrupción del torneo mediante Ctrl+C.

2. Diseño del esquema de sincronización

Para organizar el programa se separaron las funciones en distintos archivos. El módulo `matchmaker` se encarga de recibir y emparejar jugadores, mientras que los módulos `board`, `juez` y `simulacion` administran el tablero, validan movimientos, ejecutan las partidas y registran sus resultados, incluyendo la cantidad de movimientos, capturas y el estado final.

Cada partida utiliza hilos para representar al Puma y a los 12 Perros. Como las fichas pueden intentar moverse al mismo tiempo, se utilizan mutexes de POSIX para proteger el tablero. De esta manera, cuando una ficha realiza un movimiento, se bloquea temporalmente el acceso al estado compartido. Una vez terminado el movimiento, se libera el mutex para que otra ficha pueda acceder. Esto evita que dos fichas modifiquen el tablero al mismo tiempo.

Para administrar las mesas se utiliza una cola de partidas y variables de condición. Cuando todas las mesas están ocupadas, los jugadores deben esperar hasta que alguna quede disponible, sin estar consultando constantemente si se liberó una mesa.

2.1 Protección y sincronización del tablero

El tablero se protege mediante un mutex de POSIX (`pthread_mutex_t`), declarado dentro de la estructura `Tablero`. Este mecanismo permite controlar el acceso concurrente de los hilos al tablero. Antes de realizar operaciones de lectura o modificación, se utiliza `pthread_mutex_lock()` para adquirir el bloqueo y, una vez finalizada la operación, `pthread_mutex_unlock()` para liberarlo.

Este mecanismo se aplica en operaciones como mover fichas, realizar capturas, colocar fichas y consultar el estado del juego. De esta manera, se evitan condiciones de carrera y se mantiene la consistencia del tablero durante la ejecución concurrente de las partidas. Finalmente, el mutex se destruye mediante `pthread_mutex_destroy()` cuando termina su utilización.

3. Análisis de riesgos de concurrencia

Durante el desarrollo se consideraron algunos problemas que pueden ocurrir cuando varios hilos trabajan al mismo tiempo.

- **Condiciones de carrera:** pueden ocurrir cuando dos fichas intentan ocupar una misma posición. Para evitarlo se utilizan mutexes que protegen las modificaciones del tablero.
- **Deadlock:** ocurre cuando los hilos quedan bloqueados esperando recursos entre sí. Para disminuir este riesgo, se utiliza una protección central del tablero y se procura liberar los bloqueos después de cada operación.
- **Starvation:** puede ocurrir cuando una ficha debe esperar varias veces para acceder al

tablero, porque otras fichas consiguen utilizarlo primero. Los retardos aleatorios ayudan a variar el orden de los movimientos, aunque no garantizan que todas las fichas tengan las mismas oportunidades.

- **Livelock:** puede presentarse si las fichas siguen intentando moverse sin conseguir avanzar. Para disminuir este problema se utilizan retardos entre los intentos y un límite de duración de las partidas.

Ante el abandono de un jugador o una interrupción mediante SIGINT (Ctrl+C), el sistema solicita la detención de las partidas correspondientes. Para gestionar la interrupción, se utiliza un hilo encargado de esperar la señal mediante `sigtimedwait()`, que solicita el cierre ordenado del servidor. Posteriormente, el programa espera la finalización del torneo, une el hilo de monitoreo mediante `join()` y detiene el servidor.

4. Evaluación del diseño

Se decidió utilizar mutexes porque permiten controlar el acceso al tablero de una manera relativamente sencilla. Aunque esto significa que solamente un hilo puede modificar el tablero a la vez, facilita mantener un estado correcto y evitar movimientos inconsistentes.

El uso de varias mesas permite ejecutar partidas en paralelo, respetando el máximo indicado por el usuario. Esto permite atender a más jugadores sin tener que esperar a que termine todo el torneo para iniciar otra partida.

En cuanto al rendimiento, los mutexes pueden generar pequeños tiempos de espera cuando varias fichas quieren moverse al mismo tiempo. Sin embargo, para el tamaño del tablero y la cantidad de fichas del juego, se consideró una solución adecuada y más fácil de implementar.

5. Pruebas realizadas

Para comprobar el funcionamiento del programa se realizaron distintas pruebas, obteniendo los siguientes resultados:

Prueba	**Resultado**
Ejecución normal	Aprobado
Configuración personalizada	Aprobado
Detención mediante Ctrl+C	Aprobado
Abandono de jugadores	4/4 aprobados
Prueba de estrés	30/30 partidas aprobadas
Prueba de integridad	4/4 aprobados
Prueba del trono	9/9 aprobados
Configuración inválida	Rechazada correctamente

Estas pruebas permitieron revisar el funcionamiento general del programa, el comportamiento de las partidas y algunas situaciones especiales. Aunque los resultados fueron positivos, esto no significa que se puedan descartar todos los posibles errores de concurrencia.

6. Conclusión

El proyecto permitió aplicar conceptos fundamentales de Sistemas Operativos, como la administración de hilos, la sincronización y el control de recursos compartidos. Las pruebas realizadas permitieron validar el funcionamiento del sistema ante ejecuciones concurrentes, abandonos e interrupciones externas.